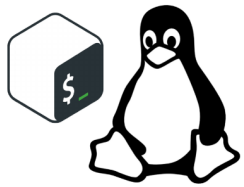
AI224: Operating Systems — Lab 9

Dr. Karim Lounis

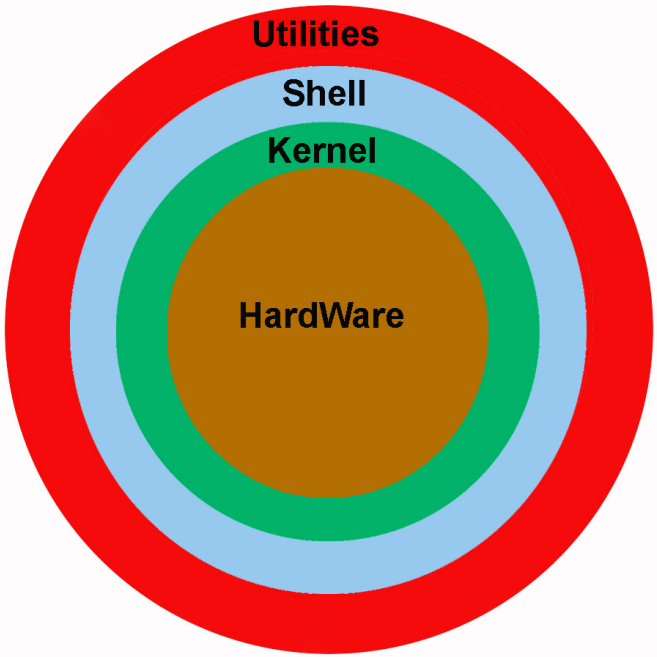
Jan - May 2025

Bash Scripting 1



Operating System's Shell

Shell. Is a program that implements the interface of interaction (the outermost layer) between the operating system (kernel) and the user.



Operating System's Shell

Shell. Is a program that implements the interface of interaction (the outermost layer) between the operating system (kernel) and the user.

- The shell uses either a command-line interface (CLI) or a graphical user interface (GUI) to allow interaction with the operating system.

  Hence, there are CLI shells and graphical shells. Certain OSs provide both.

- In the Unix world, there exist two main types of CLI shells: the Bourne shell (AT&T Bell Labs, 1970s) and the C-shell (called csh or tcsh).

  The default prompt in a Bourne-type shell is the $ sign, whereas, in a C-type shell the default prompt would be a % sign.

- The prompt (either $ or %), which is called command prompt, is issued by the shell. While the prompt is displayed, you can type a command.
- Various subcategories for Bourne Shell: Korn shell (ksh), GNU Bourne again shell (bash), and the POSIX shell.

  In Unix-like OSs, check the value of the variable `$SHELL` or `$0`.

Operating System's Shell

Shell. Is a program that implements the interface of interaction (the outermost layer) between the operating system (kernel) and the user.

- You may have several CLI shells installed on your system.
  Check the content of `/etc/shells` on your Unix system.
- When you open the Terminal app, a default CLI shell is opened:
  You can run the command `grep $USER /etc/passwd`
- A CLI shell comes with a command-line interpreter that reads user input and executes commands.
  So, it requires commands to be entered with a particular syntax.
- You can either types commands (one by one — as we've been doing so far) through the terminal (an interactive shell) or store a set of commands in a file (called a **shell script**) to be invoked as a program.
  Shell scripts are similar to batch scripts in MS-DOS shell (dosshell).
- A shell script usually takes an extension of `*.sh` or `*.csh`.

Shell Scripts

Shell Script. Is a sequence of shell and operating system commands that are stored in a file and that can be executed by a shell interpreter.

- Can be used for administrative task automation.
- Can be used to create complex and long commands.
- Add value to your operating system.
- Another programming language for you to learn.

Bash. Is a Unix shell and command language written by Brian Fox for the GNU Project as a free software replacement for the Bourne shell.

- First released in 1989. It has been used as the default login shell for most GNU/Linux distributions.
- A version is also available for Windows 10 and Windows 11 via the Windows Subsystem for Linux.

Bash Script. Is a file that contains a sequence of shell commands and instructions that can be interpreted and executed by the bash shell.

Bash Scripts

To write a bash script, you need to follow the steps below:

1. Use a text editor, e.g., `nano`, to write the code.
2. Add the `shebang` line, which is `#!/bin/bash`. This lines indicates that the file should be interpreted by a bash shell interpreter.
3. Write code inside the file following the proper syntax.
4. Save the file [with extension `.sh`, e.g., `test.sh`].
5. Add execution right to the file containing your script.
6. Run the script as `./test.sh` or `bash ./test.sh`.

Exercise. In few minutes, write a "Hello, World!" bash script:

```
$ echo '#!/bin/bash' > test.sh
$ echo 'echo Hello, World' >> test.sh
$ chmod 755 test.sh
$ ./test.sh
Hello, World
$
```

Bash Scripts

The following is an example of bash script:

```
#! /bin/bash

#This bash script displays the hello world message then runs some code

echo "Hello, World! \n"

#Here, you can add more code

mkdir -p folder_1/folder_2/folder_3

echo "Directories created \n"

echo "List of processes:\n" > folder_1/folder_2/list_of_processes.txt
echo "Date:" >> folder_1/folder_2/list_of_processes.txt
date >> folder_1/folder_2/list_of_processes.txt
echo "\n" >> folder_1/folder_2/list_of_processes.txt
ps -aux >> folder_1/folder_2/list_of_processes.txt

echo "Saved list of current processes \n"

```

Could you figure out what this script does?

Bash Scripts (Variables)

In Unix, there are two types of variables:

- **System Variables.** Defined by capital letters and are maintained by the operating system.

  E.g., HOME, USER, PATH, LANG, PWD, SHELL, LOGNAME, EDITOR, etc.

- **User Defined Variables.** A.k.a., UDVs. Created and maintained by the user. Generally, defined in lower case.
  - The name of the variable must start with an alphanumerical character or underscore followed by one or more alphanumerical character.
  - You can assign value as follow: `var_name=value`.
  - No space should be put before or after the equal sign.
  - Variables are case sensitive.
  - Special characters are not allowed when naming.
  - You retrieve the value of a variable as `$var_name`.

☞ If `myvar=10` then `echo myvar` would print?

Bash Scripts (Arithmetic)

In Unix, we define an arithmetic expression as follow:

```
expr op1 mathOp op2
```

Examples:

- `$ expr 1 + 3` (watch out, not `$ expr 1+3`)
- `$ expr 5 - 3`
- `$ expr 18 / 3`
- `$ expr 11 * 4`

To evaluate an expression, we use the following syntax:

```
`expr 2 + 3`
```

You may affect the result of an evaluation:

`var=$[2 + 3]` or `var=$((2 + 3))` or `var=`expr 2 + 3`

Bash Scripts (Standard Input/Output)

In Unix, to read input from keyboard and store it into a variable, we use:

```
read myvar
```

Example:

```
read myvar
2023
echo $myvar
2023
```

In Unix, you can display content of variables using:

`echo $myvar` or `echo ${myvar}`

Or display messages and variables:

```
echo "The content of myvar is :  $myvar"
```

Bash Scripts (Shorthands)

The following shorthands can be used to quickly search and display content:

Shorthand	**Semantics**
ls *	show all the files.
ls a*	show all files that start with an ‘a’.
ls *.c	show all files with extension .c.
ls W*.exe	show all executable files that start with a ‘W’.
ls ?	show all files whose name is one character long.
ls ???	show all files whose name is three characters.
ls [adz]*	show all files whose name start with an ‘a’, ‘d’, or ‘z’.
ls [!az]*	show all files whose name do not start with an ‘a’ or ‘z’.

And there is more ...

Bash Scripts (If Conditions)

We can express conditional statements as follows:

```
if condition_1
then
command_1
elif condition_2
command_2
elif condition_3
command_3
...
else
command_n
fi
```

The condition may need to be evaluated: [condition]:

```
if [ $choice -eq 2023 ] then shutdown -t 0 fi
```

Watch out, there is a space after the "[" and before the "]" character.

Bash Scripts (If Conditions)

We can express conditional statements as follows:

Math expression in bash	**Semantics**	**Standard**
-eq	is equal to	==
-ne	is not equal to	!=
-lt	is less than	<
-le	is less than or equal	<=
-gt	is greater than	>
-ge	is greater than or equal	=>

Bash Scripts (Loops)

We can use **for loops** as follows:

```
for { condition } in { list }
do
set of instructions
done
```

We can use **while loops** as follows:

```
while { condition }
do
set of instructions
done
```

We can also use **do while loops**.

Bash Scripts (For Loop)

Below are some examples of for loop in bash:

```
#! /bin/bash

#This is a for loop that displays ENSIA 3 times
for i in 1 2 3
do
echo "ENSIA $i times"
done

#This is an infinite for loop
for (( ; ; ))
do
echo "Hello"
done

#This is a for loop that displays ENSIA 5 times

for i in {1..5}
do
   echo "ENSIA $i times"
done
```

Bash Scripts (While Loop)

Below are some examples of for while in bash:

```
#! /bin/bash

#This is a while loop that displays ENSIA 3 times
i=1
while [ $i -lq 3 ]
do
echo "ENSIA $i times"
done

#This is an infinite while loop
while true
do
echo "Hello"
done

#This another infinite while loop that displays ENSIA

while :
do
   echo "ENSIA $i times"
done
```

Bash Scripts (Do While Loop — Until)

Below is an example of do while in bash:

```
1  #! /bin/bash
2
3  val=2
4
5  until [ $val -eq 5 ]
6  do
7   echo ENSIA
8  done
```

Bash Scripts (Case Statement)

We can use **case statement** as follows:

```
case $varname in
pattern 1) some commands ...;;
pattern 2) some commands ...;;
...) ;;
pattern x) some commands ...;;
*) some commands ...;;
esac
```

```
1  #! /bin/bash
2
3  echo -n "choose option 1, 2, or 3 "
4  read option
5
6  case $option in
7  1) echo "choice 1";;
8  2) echo "choice 2";;
9  3) echo "choice 3";;
10 *) echo "choice X";;
11 esac
```

Bash Scripts (Functions)

We can use **functions** as follows (the `function` keyword can be used):

```
function_name()   //or function function_name()
{
  ...
  body of the function
  ...
}
```

The function is called as: `function_name arg1 arg2 ... argn`

The curly braces must be separated from the body by spaces or newlines.

The single line version takes a semi-column in its last instruction.

In Bash, all variables by default are defined as global, even if declared inside the function. Use `local` for local variables inside functions.

Bash Scripts (Functions)

You can access the parameters as \$1, \$2, . . . , \$n. Generally, 9 parameters. You may need `shift n` to access beyond 9.

The \$0 variable is reserved for the function's name.

The \$♯ variable holds the number of positional parameters/arguments passed to the function.

The PID of the shell \$\$.

It is a good practice to double-quote the arguments to avoid the misparsing of an argument with spaces in it.

The symbol \$? hold the returned value of a function.

Bash Scripts (Functions)

Below is an example of function use:

```
1  #! /bin/bash
2
3  square()
4  {
5      #val=$((variable*variable))
6      val=$[variable*variable]
7      return $val
8  }
9
10 read variable
11
12 square
13
14 echo $?
```

Note that there must be a space between the operator * and the operands.

END